

Summary

This topic synthesizes Greedy Algorithm applications Optimal Merge Patterns and Huffman Encoding as presented in the GATE 2026 course. The goal in both cases is to minimize total cost: in record movements for file merges, and in total bit-length for data compression.

- Optimal Merge Pattern: Repeatedly merge the two smallest files to minimize cumulative record movements.
- Huffman Encoding: Constructs a binary tree assigning shorter codes to high-frequency characters and longer codes to low-frequency characters, ensuring the Prefix Property for unique decoding.

1. Optimal Merge Patterns

The Optimal Merge Pattern is a greedy strategy for merging n sorted files into a single sorted file using two-way merges while minimizing total record movements.

1.1 Record Movement vs. Comparison

- Sorted Arrays: Focus on comparisons ($m + n - 1$).
- Sorted Files: Focus on actual record movements; for files of size m and n , movement = $m + n$.

1.2 Greedy Strategy

1. Identify the two smallest files.
2. Merge them into a new intermediate file.
3. Replace the original files with the new file.
4. Repeat until a single file remains.

1.3 Mathematical Logic

$$\left[\begin{array}{l} \text{Total Movement} = \sum (f_i \times d_i) \end{array} \right]$$

- (f_i) : Number of records in file i
- (d_i) : Distance from leaf to root in merge tree

Key Insight: Larger files should be nearer the root, smaller files deeper in the tree to minimize total movement.

1.4 Case Study: 7-File Merge Example

Files: A: 110, B: 30, C: 90, D: 66, E: 80, F: 80, G: 50

Step	Files Merged	Resulting File Size	Remaining Files/Intermediate Files
1	B (30) + G (50)	80	110, 90, 66, 80, 80 (Inter), 80 (F)
2	D (66) + 60*	126	Continue selecting two smallest values

... ..

Total record movement for the 7-file set = 1338.

2. Huffman Encoding

Huffman Encoding applies greedy logic to variable-length encoding for data compression.

2.1 Uniform vs. Non-Uniform Encoding

- Uniform: Fixed-length codes; for 5 symbols, at least 3 bits each.
- Non-Uniform: Variable-length; shorter codes for frequent symbols, longer for infrequent symbols, reducing total bits.

2.2 Prefix Property

- No code is a prefix of another.
- Ensures unique, instantaneous decoding as each leaf is identified in the bitstream.

2.3 Constructing the Huffman Tree

1. Input: Characters with frequencies.
2. Tree Building: Merge two lowest-frequency nodes iteratively; parent node frequency = sum of children.
3. Bit Assignment: Left = 0, Right = 1 (or vice versa).
4. Code: Path from root to leaf gives binary code.

2.4 Compression Example

Characters: A:50, E:40, I:5, O:3, U:2

Character	Frequency	Fixed-Length (3-bit)	Huffman Code	Huffman Length
A	50	150 bits	0	1 bit

E	40	120 bits	11	2 bits
I	5	15 bits	100	3 bits
O	3	9 bits	1011	4 bits
U	2	6 bits	1010	4 bits
Total	100	300 bits	—	165 bits

